tangem × PESSIMISTIC

SECURITY ANALYSIS

by Pessimistic

This report is public

August 25, 2026

Abstract 3
Disclaimer 3
Summary 3
General recommendations 4
Project overview 5
Project description 5
Codebase update #1 6
Codebase update #2 6
Codebase update #3 6
Codebase update #4 6
Codebase update #5 7
Codebase update #6 8
Audit process 9
Manual analysis 11
Critical issues 11
Medium severity issues 12
M02. Fee may be unpaid (partially fixed, commented) 12
M04. Use of USDT (fixed) 12
M05. Incorrect condition for full withdrawal fee handling (fixed) 12
M06. Swap front-running (fixed) 13
M07. Merkl rewards are supplied to a suspended token (fixed) 13
M08. Fee debt can be erased without payment (fixed) 13
Low severity issues 14
L01. AToken zero address check (fixed) 14
L03. The effective balance is overestimated (fixed) 14
L04. Unindexed parameters (fixed) 14
L05. Unnecessary modifier (commented) 15
L06. Withdrawing zero-amount from Aave V3 (fixed) 15
L07. Missing initialization of ReentrancyGuardUpgradeable (fixed) 15
L08. Redundant use of require (fixed) 16
L10. Incorrect mapping key name (fixed) 16
L12. Incorrect module address on ZKsync Era (fixed) 16
L13. Deposit can be blocked by a dust transfer (commented) 17
Notes 18
N01. Project admins' roles (commented) 18

N02. Unhandled underflow (fixed) 18
N03. Swap safety checks (commented) 19
N04. Merkl rewards claimed outside of the module (commented) 19
N05. Aave pool address is bound to the implementation 20
N06. Quadratic check for duplicates (fixed) 20

ABSTRACT

In this report, we consider the security of smart contracts of Tangem project. Our task is to find and describe security issues in the smart contracts of the platform.

DISCLAIMER

The audit does not give any warranties on the security of the code. A single audit cannot be considered enough. We always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contracts. Besides, a security audit is not investment advice.

SUMMARY

In this report, we considered the security of Tangem smart contracts. We described the audit process in the section below.

The audit showed several issues of medium severity: Fee debt is not accumulating (M01), Fee may be unpaid (M02), Unhandled underflow (M03). Also, several low severity issues were found. The note on Project admins' roles (N01) was commented on by the developers. All the tests passed.

Overall code quality is good.

In the recheck #1, the codebase was updated. M01 and L02 issues were removed as false positives. M03 was commented on and moved to notes (Unhandled underflow (N02)). Fee may be unpaid (M02) issue was partially fixed and commented on. The rest of the low-severity issues and notes were fixed or commented on. One low-severity issue L06 was found and fixed by the developers.

In the recheck #2, the developers found and fixed the Use of USDT (M04) issue, the actual commit is in the Codebase update #2.

In the recheck #3, the codebase was updated again. We found two medium severity issues: Incorrect condition for full withdrawal fee handling (M05) and Swap front-running (M06). Also, several low severity issues and one note were found. All tests on the code update 3 passed.

In the recheck #4, the codebase was updated. The developers fixed two medium severity issues: Incorrect condition for full withdrawal fee handling (M05) and Swap front-running (M06). The remaining low severity issues and notes were fixed or commented on.

In the recheck #5, the codebase was updated again. The developers implemented the claiming of Merkl incentives, the risk service actions, and the support of the native token.

We found two medium severity issues:
Merkl rewards are supplied to a suspended token (M07) and
Fee debt can be erased without payment (M08). Also, several new low severity issues were found. Two new notes were added in the notes section. The list of the project admins' roles was updated. All tests passed.

In the recheck #6, the codebase was updated again. The developers fixed both medium severity issues: Merkl rewards are supplied to a suspended token (M07) and
Fee debt can be erased without payment (M08). All the low severity issues were fixed or commented on by the developers. The L11 issue was excluded from the report, as it concerned the internal documentation, and L09 was moved to the notes. All tests passed.

GENERAL RECOMMENDATIONS

We do not have any further recommendations.

PROJECT OVERVIEW

Project description

For the audit, we were provided with Tangem project on a public GitHub repository, commit 2a5c3819189e1f5281984f18c7af03461cacf8a6.

The scope of the audit included:

- **TangemAaveV3YieldModule.sol**;
- **TangemYieldModuleFactory.sol**;
- **TangemYieldProcessor.sol**;
- **YieldModuleLiquidUpgradeable.sol**;
- dependencies.

The documentation for the project included **Tangem_Yield_Module_Contracts_Functional_Requirements.pdf** (sha1sum `a09de4bbd0f6031d027fad8e60d8cd2deee8d91d`).

The list of networks for deployment:

- Ethereum;
- Avalanche;
- Arbitrum;
- Optimism;
- Base;
- Gnosis;
- Scroll;
- BSC;
- ZKsync Era;
- Polygon POS;
- Sonic.

All 73 tests pass successfully. The code coverage is 94.08%.

The total LOC of audited sources is 436.

Codebase update #1

For the recheck, the developers added the code to the new public repository on commit f685229ee1f83e5f95122005d8dd66ce17b3b9f8.

All the 73 tests passed. The code coverage was 93.18%.

One issue of medium severity was removed as false positive. Another medium severity issue was partially fixed and commented on, and the third one was moved to notes. The developers also fixed or provided comments for low-severity issues and notes. Also, they found and fixed the new low-severity issue.

Codebase update #2

In the recheck, the developers found and fixed one medium severity issue. Provided commit is 945252b2662397a4c5fa50415115e3f59565fc83.

Codebase update #3

For the recheck, the developers provided us with a new commit c3dce0d9dcf29196dcd2d3f5aa697dab79eb7fd4.

The developers provided another file with updated documentation **Functional Requirements for Tangem Yield contracts modification.pdf** (sha1sum `89bae7bcc74686dba07e55d37930951489a65950`). Also, the developers added the new **SwapExecutionRegistry** contract to the scope.

Two medium severity issues, several low severity issues, and one note were found during the audit.

All 157 tests passed. The code coverage was 97.75%.

Codebase update #4

For the recheck, the developers provided a new commit a457fadfe6e59460e2d36db508fe813685567e70.

All issues were fixed or commented on.

The number of tests did not change, and the code coverage increased to 97.85%.

Codebase update #5

For the recheck, the developers provided a new commit b51936fe8afdbe789c5aaea07e4e212778c389e0.

In this update, the developers implemented the claiming of Merkl incentives, the risk service actions for partial exits and deposit suspension, and the deposits and withdrawals in the native token. The codebase was also refactored.

The developers added the new **MerklIncentives.sol** and **NativeSupport.sol** contracts to the scope.

The documentation for the new functionality included three files:

- **Merkl Incentives DOC.pdf** (sha1sum `86ef97e962152d5ab2d47d63c2358fae08a3d925`);
- **Risk Service PRD.pdf** (sha1sum `23849176e25c313674adc7d805cdee10e4ae06d8`);
- **TRD_Native deposits and withdrawals in YM.pdf** (sha1sum `90f29e9d2e4301b098c5d777f6286b8d62954626`).

The project is already deployed on the following networks:

- Ethereum;
- BNB Smart Chain;
- Polygon POS;
- Arbitrum One;
- Base;
- Optimism;
- Avalanche.

The developers also plan to deploy the project on Monad.

All tests passed: 334 tests in Foundry and 213 tests in Hardhat. The code coverage was 100%.

Two medium severity issues, several low severity issues, and two notes were found during the recheck. The note on project admins' roles was updated.

Codebase update #6

For the recheck, the developers provided a new commit 0968615fcff3cfcc0c1c1a859777b2af85f0d43c.

The developers also clarified that the project will not be deployed on Gnosis, Scroll, ZKsync Era, and Sonic.

All tests passed: 341 tests in Foundry and 213 tests in Hardhat. The code coverage was 100%.

All the issues were fixed or commented on.

AUDIT PROCESS

We started the audit on September 10 and finished on September 16, 2025.

We inspected the materials provided for the audit. During the work, we stayed in touch with the developers and discussed confusing or suspicious parts of the code.

We manually analyzed all the contracts within the scope of the audit and checked their logic. Among other things, we verified the following properties of the contracts:

- Standard Solidity issues;
- Integration with Aave V3;
- Whether fees are correctly accounted for and whether they could be bypassed.

We scanned the project with the following tools:

- Static analyzer Slither;
- Our plugin Slitherin with an extended set of rules;
- Semgrep rules for smart contracts;
- AI tool Savant.Chat;
- AI tool AuditAgent.

We ran tests and calculated the code coverage.

We combined all the verified issues we found during the manual audit or discovered by automated tools in the private report.

We performed the recheck on September 23-26, 2025. One issue of medium severity was removed as false positive. Another medium severity issue was partially fixed and commented on, and the third one was moved to notes. The developers fixed or commented on low-severity issues. They also discovered and fixed L06.

We also re-ran the tests and recalculated code coverage. Finally, we updated the report.

We rechecked the fix for issue M04 on November 3, 2025, after the developers identified and implemented a fix for the bug. The report was updated accordingly.

We performed the recheck #3 from March 20 to 24, 2026. During this period, we reviewed the new swap logic and the new registry contract, re-ran the tests, and recalculated code coverage.

We scanned the project with the following tools:

- AI tool Savant.Chat;
- AI tool AuditAgent.

The report was updated accordingly.

We performed the recheck #4 on March 27, 2026. We verified the fixes, re-ran the tests, and recalculated the code coverage.

We performed the recheck #5 from August 18 to 21, 2026. During this period, we reviewed the new functionality and the diff between the commits, re-ran the tests, and recalculated code coverage.

We verified the following properties of the contracts:

- Integration with Merkl: the claiming, the fee charging, and the routing of the rewards;
- Standard Solidity issues;
- Whether the risk service actions can be bypassed and whether the entry suspension is enforced on every deposit path;
- Wrapping and unwrapping of the native token;
- Whether the refactoring preserved the fee accounting logic;
- Access control of the new roles.

We scanned the project with the following tools:

- AI tool Savant.Chat;
- AI tool AuditAgent.

The report was updated accordingly.

We performed the recheck #6 on August 25, 2026. We verified the fixes, re-ran the tests, and recalculated the code coverage.

The report was updated accordingly.

MANUAL ANALYSIS

The contracts were completely manually analyzed, their logic was checked. Besides, the results of the automated analysis were manually verified. All the confirmed issues are described below.

Critical issues

Critical issues seriously endanger project security. They can lead to loss of funds or other catastrophic consequences. The contracts should not be deployed before these issues are fixed.

The audit showed no critical issues.

Medium severity issues

Medium severity issues can influence project operation in the current implementation. Bugs, loss of potential income, and other non-critical failures fall into this category, as well as potential problems related to incorrect system management. We highly recommend addressing them.

M02. Fee may be unpaid (partially fixed, commented)

According to the **YieldModuleLiquidUpgradeable** contract, if the owner withdraws all funds from the yield module and some fees have not been paid due to an error or if the owner revoked approval for yield tokens (during `exitProtocol`), they are recorded in `feeDebts`. If the owner no longer interacts with Aave V3, these `feeDebts` in **YieldModuleLiquidUpgradeable** will remain unpaid indefinitely.

> `Comment from the developers:`
> There is no way for a user to avoid fee, the previous possibility is fixed. If the fee won't be paid for some other reason, the fee funds will stay at the module, and can be retrieved by Tangem later. The priority is to make sure our users can use their funds, even if fee payment fails, not our fee.

> The issue has been partially fixed in the `YieldModuleLiquidUpgradeable.send` function and commented on by the developers.

M04. Use of USDT (fixed)

The `TangemAaveV3YieldModule._pushToProtocol` function performs an approve call for Aave using the **ERC20** interface. However, when the token is USDT, the transaction reverts because the USDT implementation does not return a boolean value from approve, while the **ERC20** interface expects it.

> The issue has been found and fixed by the developers.

M05. Incorrect condition for full withdrawal fee handling (fixed)

In the `YieldModuleLiquidUpgradeable.send` function at line 169, the `protocolBal == amount + fee` check is used to detect the full withdrawal case.

Using `amount` in this comparison may lead to incorrect fee calculation, which may result in collecting more than just the intended rounding remainder. Consider using `pullAmount` instead.

> The issue has been fixed and is not present in the latest version of the code.

M06. Swap front-running (fixed)

The `YieldModuleLiquidUpgradeable.finalizeSwap` function requires `tokenIn.balanceOf(address(this)) == 0` after a swap. An attacker can front-run the `swap` or `swapAndReceive` functions by sending 1 wei of `tokenIn` to the contract, causing the check to fail and reverting the transaction, effectively blocking all swaps.

> The issue has been fixed and is not present in the latest version of the code.

M07. Merkl rewards are supplied to a suspended token (fixed)

The `MerklIncentives._classifyRewardRoute` function determines the route of a claimed reward using only the `active` flag of the yield token and does not check `entrySuspended`.

As a result, a token that is active but suspended for new entries is routed as `PUSH_TO_PROTOCOL`, and `_routeClaimedReward` supplies the reward to Aave V3. This bypasses the risk service control when the pool is considered unsafe.

Consider using the `_isEntryAllowed` check, which requires the token to be both active and not suspended.

> The issue has been fixed and is not present in the latest version of the code.

M08. Fee debt can be erased without payment (fixed)

The `YieldModuleLiquidUpgradeable._withdrawAndDeactivate` function passes the fee value capped by the protocol balance to `_processFeeAfterProtocolPull` instead of the complete one.

When the protocol balance is zero and `feeDebts` is not, the recalculated amount equals zero at line 238 of the **YieldModuleLiquidUpgradeable** contract, and `_tryProcessFee` of the **FeeAccounting** contract treats a zero amount as a successful payment at line 35. Therefore, `feeDebts` is set to zero and the debt is erased without transferring any tokens.

The owner can reach this state by calling `reactivateToken` and `withdrawAndDeactivate` after `exitProtocol` recorded an unpaid debt.

> The issue has been fixed and is not present in the latest version of the code.

Low severity issues

Low severity issues do not directly affect project operation. However, they might lead to various problems in future versions of the code. We recommend fixing them or explaining why the team has chosen a particular option.

L01. AToken zero address check (fixed)

The `YieldModuleLiquidUpgradeable.initYieldToken` function does not verify the **aToken** address returned by `_initProtocolToken`. If a yield token is not supported by Aave V3, the getReserveData returns `address(0)`.

This does not impact security because `pool.supply` in Aave V3 already checks the token validity. But an early check of the **aToken** address can be added to prevent unnecessary gas consumption when the token is unsupported.

> The issue has been fixed and is not present in the latest version of the code.

L03. The effective balance is overestimated (fixed)

The `YieldModuleLiquidUpgradeable.effectiveBalance` function can overestimate the actual available balance, as it does not account for `feeDebts`.

It is also important to handle the underflow case when the sum of `_calculateServiceFee` and `feeDebts` exceeds the owner's and protocol's balances.

> The issue has been fixed and is not present in the latest version of the code.

L04. Unindexed parameters (fixed)

Consider declaring parameters of the `YieldModuleDeployed` event as `indexed` to enhance off-chain filtering in the **TangemYieldModuleFactory** contract.

> The issue has been fixed and is not present in the latest version of the code.

L05. Unnecessary modifier (commented)

The `whenNotPaused` modifier does not affect the security of the `TangemYieldModuleFactory.calculateYieldModuleAddress` function because the function is declared as `view` and only pre-calculate the **YieldModule** address.

> `Comment from the developers:`
> This modifier is added to avoid unnecessary approvals from the client. If the contract is paused, the implementation can change, leading to the future module address changing. If the client software will do approval as a first step without checking, it will give the approval to a potentially incorrect address, wasting funds for network fee. With the pause, the deployment cannot be completed anyway, so there is no real reason to allow this function usage during pause.

L06. Withdrawing zero-amount from Aave V3 (fixed)

The `YieldModuleLiquidUpgradeable.withdrawAndDeactivate` function reverts if `amountToExit` is equal to 0, since this value is passed to Aave V3 `pool.withdraw`, which reverts on zero amounts. As a result, `yieldTokensData` will not be updated.

> The issue has been fixed and is not present in the latest version of the code.

L07. Missing initialization of ReentrancyGuardUpgradeable (fixed)

The **YieldModuleLiquidUpgradeable** contract inherits from **ReentrancyGuardUpgradeable** and uses the `nonReentrant` modifier in the `swap` and `swapAndReceive` functions but __ReentrancyGuard_init is never called during initialization.

As a result, initializing status remains 0 (default value) instead of the expected initialized state (`_NOT_ENTERED`). While this is not currently critical due to the existing implementation, it may lead to issues if OpenZeppelin changes the internal logic in future versions.

> The issue has been fixed and is not present in the latest version of the code.

L08. Redundant use of require (fixed)

In the `YieldModuleLiquidUpgradeable.withdrawAndDeactivate` function at line 205, the `require(protocolBal >= fee, InsufficientFunds())` is redundant. At the next line 206, the code checks whether `protocolBal >= fee` and sets `amountToExit = 0` otherwise.

Since the `require` already reverts when `protocolBal < fee`, this condition can never be true, making the assignment of 0 meaningless.

This also prevents the user from withdrawing and deactivating, if the user has large debt, which contradicts the comment at line 206.

> The issue has been fixed and is not present in the latest version of the code.

L10. Incorrect mapping key name (fixed)

In the **YieldModuleStorage** contract, the key of the `isProtocolToken` mapping is named `yieldToken`. However, this mapping stores the flags of protocol tokens.

Named mapping parameters do not affect the behavior of the code, but consider declaring the key as `protocolToken` to avoid misleading and improve code readability.

> The issue has been fixed and is not present in the latest version of the code.

L12. Incorrect module address on ZKsync Era (fixed)

The `TangemYieldModuleFactory.calculateYieldModuleAddress` function pre-calculates the module address using the standard `CREATE2` formula. ZKsync Era derives contract addresses in a different way, so this function will return an incorrect address on this network. The client software may therefore approve the tokens to the wrong address.

The project is not deployed on ZKsync Era at the moment, but this network is still present in the developers' plans.

> `Comment from the developers:`
>
> Acknowledged. Not applicable to the supported deployment scope.

> `Comment from Pessimistic:`
>
> ZKsync Era was removed from the list of networks targeted for deployment in Codebase update #6.

L13. Deposit can be blocked by a dust transfer (commented)

The `YieldModuleLiquidUpgradeable._enterProtocol` function treats all available funds as the entire token balance of the owner and does not limit it by the allowance granted to the module.

If the owner approves exactly their current balance, an attacker can front-run the deposit and transfer one wei of the same token to the owner. The module then requests the increased balance, which exceeds the allowance, and the transfer reverts.

The same applies to `resumeAndEnterProtocol`, where the failed deposit also rolls back the clearing of the suspension flag, so the risk service cannot restore the entry for the token.

Consider using the minimum of the owner's balance and the allowance granted to the module.

> `Comment from the developers:`
>
> By design, the supported client flow grants the module an unlimited token allowance (`type(uint256).max`). Therefore, a dust transfer cannot cause the owner's balance to exceed the granted allowance under the supported flow.

Notes

N01. Project admins' roles (commented)

In the current implementation, the system depends heavily on the admin role. Thus, there are scenarios that can lead to undesirable consequences for the project and its users, e.g., if admin's private keys become compromised. The description of admins' roles:

- The `IMPLEMENTATION_SETTER_ROLE` can change the implementation in the **TangemYieldModuleFactory** contract;
- **TangemYieldProcessor** contract:
  - The `PAUSER_ROLE` can pause/unpause contract and influence on the operations, including setting service fee and fee receiver addresses, collecting service fee, entering/exiting protocol;
  - The `PROTOCOL_ENTERER_ROLE` can supply to Aave V3 and create or increase owner's position;
  - The `PROTOCOL_EXITER_ROLE` can close owner's position by calling `exitProtocol` and withdrawing funds from Aave V3;
  - The `PROPERTY_SETTER_ROLE` can change the service fee rate and fee receiver, and the `SERVICE_FEE_COLLECTOR_ROLE` can withdraw all fees to the fee receiver;
  - The `CLAIM_MERKL_REWARDS_ROLE` can claim Merkl rewards for any yield module;
  - The `RISK_SERVICE_ROLE` can suspend the deposits, perform full or partial exits from the protocol by calling `softExit`, and resume the entry by calling `resumeAndEnterProtocol`.

We recommend designing contracts in a trustless manner or implementing proper key management, e.g., setting up a multisig.

> According to the developers' comment, they will use multisig to manage the project. In the recheck #5, two more roles were added to the **TangemYieldProcessor** contract.

N02. Unhandled underflow (fixed)

The `YieldModuleLiquidUpgradeable.withdrawAndDeactivate` may revert due to underflow at line 155 if the `feeDebts` exceeds the protocol balance. This can lock the owner's funds until the `exitProtocol` function is called by the processor or `_enterProtocol` is used to increase the protocol balance.

> The issue has been fixed and is not present in the latest version of the code.

N03. Swap safety checks (commented)

Although the project uses its own data service for swaps, the **YieldModuleLiquidUpgradeable** contract executes swaps via the `_callProvider` function using `target.call{value: msg.value}(data)`, so `target` and `data` must be set correctly.

And there is no check on `amountOut` in the `_finalizeSwap` function, which could make swaps vulnerable to slippage or sandwich attacks. Consider verifying `amountOut` or using a router that enforces a minimum received amount.

> `Comment from the developers:`
>
> There is no way to provide the same level of liquidity without the use of external swap data sources. We will be using our own swap aggregator service with only well-proven providers. Additional `amountOut` check is impossible in some instances (swap with bridge to other chain), and redundant in other, since the router or pool already perform the same check. It will not only add complexity, but additional costs for the end user.

N04. Merkl rewards claimed outside of the module (commented)

The **MerklIncentives** contract charges the service fee in the `_takeServiceFee` function, which is only reached through `_claimMerklRewards`. According to the Merkl documentation:

"Admin rights to claim for everyone: In some cases, certain addresses may be granted admin rights that allow them to claim rewards on behalf of all users without requiring individual operator approvals. These admin rights are granted on a case-by-case basis and enable streamlined reward distribution for specific use cases, such as protocol-level reward forwarding or automated claim processes".

If such a claim is performed, the rewards are sent directly to the module address bypassing `_claimMerklRewards`, so the service fee is not charged. If the reward token is neither the protocol token nor a yield token, the owner can then withdraw the entire amount via `withdrawNonYieldToken` without paying the service fee.

> `Comment from the developers:`
>
> Acknowledged. By design. According to the Merkl team, “admin/autoclaim” transactions account for no more than 1% of the total number of requests. We are prepared to assume the limited risk associated with lost revenue in the form of service fees on such externally initiated requests. This does not affect user funds and represents only a potential loss of revenue from protocol fees.

N05. Aave pool address is bound to the implementation

The `pool` variable of the **TangemAaveV3YieldModule** contract is declared as `immutable`, so it is stored in the bytecode of the implementation rather than in the storage of the proxy. At the same time, `YieldModuleBase._authorizeUpgrade` only checks that the new implementation is the current one of the factory and does not compare its pool address with the pool of the existing position.

Therefore, if the owner upgrades the module to an implementation bound to another pool and approved by the factory, the module keeps the aTokens of the old pool in the `protocolTokens` mapping, while all withdrawals start addressing the new pool. As a result, both the owner withdrawals and the emergency exits performed by the processor will revert, and the position stays frozen until the module is upgraded back to an implementation bound to the original pool.

N06. Quadratic check for duplicates (fixed)

The `MerklIncentives._claimMerklRewards` function has a gas-inefficient check for duplicate reward tokens at line 71. Therefore, its complexity is quadratic in the number of reward tokens.

Consider reducing the quadratic complexity to linear by passing the sorted list of the reward tokens to the contract.

> The issue has been fixed and is not present in the latest version of the code.

This analysis was performed by Pessimistic:

Daria Korepanova, Senior Security Engineer
Yhtyyar Sahatov, Security Engineer
Irina Vikhareva, Project Manager
Alexander Seleznev, CEO

August 25, 2026